

BUSINESS WHITEPAPER

flydocs

Pure-multimodal intelligent document processing
for the production back-office.

A single HTTP call that turns any document into structured, validated, audit-ready data — with bounding boxes, deterministic checkers, LLM cross-examination, and a business-rule engine, ready to drop into KYC, KYB, claims, invoice, onboarding and operations pipelines.

Edition	v1.0 · May 2026
Audience	CIOs, CTOs, heads of operations & engineering, IDP buyers
Product	flydocs — part of Firefly OperationOS
License	Apache-2.0 (service core), proprietary plugins available
Repository	github.com/firefly-operationOS/flydocs

Prepared by Firefly Software Solutions Inc.
Copyright © 2026 Firefly Software Solutions Inc. All rights reserved.

Table of contents

Executive summary	3
The state of intelligent document processing	5
Introducing flydocs	7
The pure-multimodal approach	9
Architecture overview	11
The pipeline	14
Binary normalization	17
Extraction and bounding-box grounding	18
Validation, authenticity, and the judge	20
Post-extraction transformations	22
The business rule engine	23
Asynchronous jobs and event-driven backbone	25
Observability, cost telemetry, and prompt caching	27
Deployment and security	29
Extensibility and the Firefly Framework	31
Use cases	33
Competitive positioning	34
Roadmap and direction of travel	35
Economics and operational profile	36
Conclusion	38

CHAPTER 00

Executive summary

Every back-office workflow that touches documents — KYC, KYB, claims triage, invoice intake, vendor onboarding, mortgage underwriting, notarial review — reduces to the same underlying loop: *read the document, decide what it says, decide whether it passes our checks, and route it accordingly*. Classical intelligent document processing (IDP) stacks solve this with OCR pipelines, hand-coded templates, and a brittle string of rules that breaks every time a vendor redesigns a form.

flydocs collapses the whole workflow into a single HTTP call. The caller declares the fields and rules they care about as JSON; the service returns a structured verdict — every value tagged with a bounding box, a confidence score, a validation outcome, an independent LLM judge re-check, and the resolved business rules. No layout templates, no OCR coordinates, no model fine-tuning, no per-vendor pipeline.

The service is built on the **Firefly Framework** (*fireflyframework-pyfly* for the service runtime and *fireflyframework-agentic* for the multimodal agent runtime). It runs as a Python 3.13, async-first application exposing both a synchronous API (`POST /api/v1/extract`) and a queue-backed asynchronous API (`POST /api/v1/jobs`) with HMAC-signed webhooks, a durable Postgres event bus, OpenTelemetry traces, Prometheus metrics, structured cost telemetry, and Kubernetes-grade health probes.

1

HTTP call from raw bytes to a routed decision

15

Pipeline stages — all but two are caller-toggled

≈90%

Of bounding boxes grounded without an LLM call

4

First-class LLM providers (Anthropic, OpenAI, Google, Bedrock)

32

Built-in deterministic field validators (IBAN, NIF, IBAN, ...)

3

Pluggable OCR engines (Tesseract, Docling layout-aware, none)

Why flydocs wins. Three properties distinguish it from first-generation IDP platforms and from rolling-your-own on top of a multimodal model.

First — pure multimodal. Documents go to the LLM as native bytes (PDFs, images), not as OCR text. The model reads layout, signatures, stamps, and handwriting the same way a human does. There are no OCR templates to maintain, no per-document fine-tuning, and no layout-engineering team.

Second — verifiable by construction. Every extracted value carries a bounding box, a confidence score, and a deterministic validation verdict (IBAN mod-97, NIF mod-23, Luhn, ISO date, ...). A second LLM pass — the *judge* — re-grades every value against the source document and stamps PASS / FAIL / UNCERTAIN with quoted evidence. A business-rule engine evaluates caller predicates as a DAG, returning decisions (approve, escalate, reject) that the upstream workflow can act on without a human in the loop.

Third — production-grade by default. The service ships with W3C trace propagation, RFC 7807 error responses, idempotent submission, HMAC-signed webhooks, durable async jobs, prompt caching across every supported provider, per-call cost telemetry, and a clean failure-isolation model — a single failed stage

degrades to a partial result instead of failing the whole request.

Positioning

flydocs is product- and vertical-agnostic. It is the document intelligence module of Firefly OperationOS, but it ships as a standalone HTTP service that can drop into any platform that needs to turn unstructured documents into verified, structured, decisioned data.

CHAPTER 01

The state of intelligent document processing

Document automation is one of the oldest unsolved problems in enterprise software. Despite three decades of OCR engines, template editors, RPA bots, and managed cloud services, every operations team that touches paperwork still ends up hand-stitching parts of the pipeline. The reasons are now well understood.

Why traditional IDP pipelines break

First-generation IDP stacks share an architecture: rasterise the input, run an OCR engine over each page, locate the fields with a layout template, normalise the values, and call the downstream system. The architecture is brittle along four dimensions that compound in production.

Pressure point	What happens in practice
Layout drift	A supplier redesigns a form. The OCR coordinates no longer match. The template editor team queues a fix. The pipeline falls back to manual review until the template ships.
New document types	Operations onboards a new partner with a new document. Building a template takes days or weeks. The pipeline cannot accept it until the template lands.
Handwritten or signed content	OCR sees the printed labels but misses the handwritten cells, ticked checkboxes, signatures, stamps, and crossed-out values that carry the document's actual meaning.
Document quality variance	Phone photos, scanner skew, low-DPI scans, multi-page bundles, and rotated pages each require their own pre-processing branch. Maintaining them is a full-time job.
Vertical lock-in	Vendors that solve one vertical (invoices, IDs, claims) usually encode that vertical in their data model. Extending to a second vertical is a full integration project rather than a configuration change.

The multimodal-LLM opportunity

Modern multimodal large language models (Anthropic Claude, OpenAI GPT-4o, Google Gemini, the Bedrock-hosted Anthropic models) read documents end-to-end. They see layout, handwriting, signatures, stamps, table structure, and the relationships between fields. A schema-driven prompt is enough to coax structured JSON output — no template editor, no layout training, no per-vendor coordinates. For most document classes the accuracy ceiling has moved from a four-week template engineering project to a two-line schema declaration.

The catch is that the LLM, on its own, is not a production system. It hallucinates plausible values; it cannot verify a checksum; it does not know whether the dates and totals on a document actually add up; it has no opinion on whether the result satisfies a caller's compliance rule. Wired naively, a raw multimodal extractor produces output that looks right but fails in subtle ways that surface in regulator audits and customer disputes weeks later.

The integration gap

Every team that has shipped a multimodal extractor into production knows the same set of follow-up problems. The extractor returns plausible JSON, but no field carries a bounding box — so a reviewer cannot locate the value on the page. The output is not deterministically validated, so an invalid IBAN passes through. There is no second model verifying the first, so hallucinations escape into downstream systems. There is no business-rule layer, so the decision logic ends up in the caller's codebase, where each team rebuilds it from scratch. There is no async story, no webhook protocol, no idempotency, no cost telemetry, no observability, no failure isolation — and shipping any one of those properly is a quarter of work per team.

flydocs is the layer that closes the integration gap. It treats the multimodal LLM as a primitive, surrounds it with the disciplines an enterprise back office requires, and exposes the result as one HTTP call.

“Take this document, tell me what it says, decide whether it passes our checks, and route it accordingly.” The promise of an IDP service is to make that sentence a single HTTP request, with the verdict signed for audit.

CHAPTER 02

Introducing flydocs

flydocs is a pure-multimodal IDP service. The caller submits one or more documents and a declarative description of what they care about. The service returns a structured verdict covering every field, every validation, every authenticity check, every business-rule output, and a full audit trail. The same API works synchronously and as a queued asynchronous job with an HMAC-signed webhook.

The contract, in one shape

Every request carries three pieces of information. The *documents* array contains one or more base64-encoded files; the *docs* array contains one *DocSpec* per document type the caller expects, each declaring the field schema, the visual validators, and the deterministic *standard validators* to run; and the *rules* array contains the caller-defined business predicates that produce decisions over the extracted data.

```
{
  "intention": "KYC review for a Spanish power-of-attorney deed.",
  "documents": [
    {
      "filename": "deed.pdf",
      "content_base64": "JVBERi0xLjQK...",
      "content_type": "application/pdf"
    }
  ],
  "docs": [ /* one DocSpec per expected document type */ ],
  "rules": [ /* declarative business rules */ ],
  "options": {
    "stages": { "judge": true, "rule_engine": true,
    "bbox_refine": true, "transform": true }
  }
}
```

What you get back

The response mirrors the request. For every document in the submission you receive: the extracted fields with their values, page numbers, normalised bounding boxes, model confidence, and free-text notes; the verdict of every deterministic field validator; the outcome of every visual authenticity check; a document-level content integrity audit; the second-pass judge verdict with quoted evidence; the post-extraction transformation outputs; and the resolved business-rule decisions. Every response also carries a *usage* block with aggregated token counts and estimated USD cost, an *execution trace* with per-stage latency, and an *outbound_call* log line for every external call the service made on the request's behalf.

Layer	What it tells the caller
Fields	Extracted value, page number, normalised bounding box (with a geometric quality verdict, a source discriminator — <i>llm</i> / <i>pdf_text</i> / <i>ocr</i> — and a refinement confidence), model confidence, and free-text notes. Array fields nest recursively.
Field validation	Per-field PASS / FAIL plus the verdict of every built-in StandardValidator (IBAN, BIC, NIF/NIE/CIF, VAT, SSN, Luhn, phone E.164, country / language / postal code, lat/lon, IPv4/IPv6, URI/URL/email/domain/slug, UUID, JSON, hex colour, date / time / datetime, currency code, amount, passport).
Visual authenticity	Yes/no verdicts on caller-defined visual checks: signature present, stamp present, MRZ band present, photo present, ...

Layer	What it tells the caller
Content authenticity	Document-level integrity audit: date consistency, totals tally, expected boilerplate, tampering signals.
Judge	A second LLM pass re-checks each value against the document and stamps PASS / FAIL / UNCERTAIN with evidence and a flag_for_review bit.
Judge escalation	Re-runs extract + judge with a stronger escalation_model when the judge's first pass fails too many fields.
Transformations	Declarative entity resolution (dedupe people across documents) and free-form LLM transforms.
Business rules	Boolean / categorical decisions over fields, validators, and other rules — evaluated as a DAG.
Multi-file summary	One entry per input file (filename, MIME type, page count, byte size, final document_type, classifier verdict).
Audit trail	request_id, per-stage latencies, per-doc model used, outbound_call log lines, W3C trace context propagated end to end.
Cost telemetry	Aggregated USD cost via genai-prices, broken down by agent and by model, plus per-call cost on every outbound_call log line.
Prompt caching	Provider-aware caching across Anthropic, OpenAI, Google, Bedrock, and Azure-OpenAI. Cache writes and reads surface on the response.

The two surfaces

Both surfaces share the same request shape, the same pipeline, and the same response. The choice is one of operational fit, not of capability.

Surface	When to use
POST /api/v1/extract	Synchronous. Blocks until the pipeline finishes or FLYDOCS_SYNC_TIMEOUT_S elapses (default 60 s). Right for interactive flows: a UI uploads a document and waits for the structured verdict, or a backend service performs a single inline extraction during a request handler.
POST /api/v1/jobs	Asynchronous. Returns 202 + a job id; the worker drives the same pipeline and posts the result to a caller-supplied callback URL with an HMAC-SHA256 signature. Right for batch workloads, multi-file packs, and any flow where the caller does not want to hold a request open.
POST /api/v1/extract:validate	Dry-run the semantic validator on a payload without issuing any LLM call. Useful from CI pipelines, UIs, and rule-authoring tools.
GET /api/v1/jobs/{id}/result	Long-poll for grounded bounding boxes with ?wait_for_bboxes=true&timeout;=... so callers can block until the secondary bbox-refine worker finishes grounding coordinates.

CHAPTER 03

The pure-multimodal approach

Pure multimodal means the document goes to the model as the model wants to see it — bytes, not transcribed text. The consequences run through the entire architecture, the operational model, the cost profile, and the upgrade path.

Three architectures compared

Approach	Strengths	Where it breaks
Classical OCR + templates	Mature ecosystem. Deterministic for known layouts. Predictable cost.	Template per layout, per language, per vendor. Brittle to redesigns. Bad at handwriting, signatures, stamps. New document types require weeks of engineering.
Fine-tuned vision model	Higher accuracy than OCR templates on a single closed task. Captures handwriting and table structure.	Training set per task, per language. Retraining cycle. Costly experts. Model lifecycle locked to the vendor that trained it.
Pure multimodal LLM (flydocs)	Schema-driven JSON. Handles any layout the LLM has seen at training time, plus arbitrary new variants. Zero-shot for new document types. Model provider can be swapped per request.	Per-call LLM cost. Hallucination risk if shipped raw (addressed by the validation, judge, and rule layers below).

What pure multimodal unlocks

Layout-agnosticism. The LLM does not need to be told where on the page a field sits. It locates the value the way a human reader does — and returns a bounding box that flydocs later grounds against the document's real text layer for sub-pixel accuracy.

Multilinguality without translation. Multimodal LLMs read every script their training set covered. A flydocs deployment serves Spanish notarial deeds, German lease agreements, French invoices, and English claims forms with the same prompts and the same field schema — no per-language extraction pipeline.

Provider portability. flydocs is a thin shell around the *fireflyframework-agent* abstraction, which is itself a thin shell over pydantic-ai. Switching from Anthropic Claude to OpenAI GPT-4o, Google Gemini, or a Bedrock-hosted variant is a configuration change — FLYDOCS_MODEL=anthropic:claude-sonnet-4-6 becomes FLYDOCS_MODEL=openai:gpt-4o. The fallback model can target a different provider entirely, giving the service a cross-provider failover lane out of the box.

Cheap-by-default with escalation. Production deployments run a cheap model (Haiku, gpt-4o-mini) for the first pass and a stronger model only when the judge flags too many fields. The *judge_escalation* stage automates that policy — the operator sets a failure-rate threshold and an escalation model id, and flydocs re-runs extract + judge with the stronger engine when the threshold is breached, keeping the better result.

The disciplines pure multimodal needs

The shift to pure multimodal moves the engineering work from *making the model see the page* to *making the system trust what the model said*. flydocs answers each trust-related question with a dedicated, optional pipeline stage:

Trust question	flydocs answer
Did the model put the value on the right spot of the page?	The geometric bbox validator runs on every request. Bbox refinement grounds LLM coordinates against PyMuPDF text or OCR words for sub-pixel accuracy.
Is the value syntactically correct?	Pure-Python StandardValidators run after extraction (IBAN mod-97, NIF mod-23, Luhn, ISO dates, country-aware postal codes, and 25+ others). No LLM call.
Did the model hallucinate this value?	The Judge — a separate LLM pass — re-checks every extracted value against the document and stamps PASS / FAIL / UNCERTAIN with the exact quoted evidence and a flag_for_review bit.
Is the document itself consistent and untampered?	The content authenticity stage performs a coherence audit (dates tally, totals add up, expected boilerplate, tampering signals).
Are the visible artefacts that the caller expects present?	The visual authenticity stage evaluates caller-defined yes/no checks: signature present, stamp present, MRZ band present, photo present, ribbon intact, ...
Does the extracted data clear the business decision?	The rule engine evaluates a DAG of caller predicates and returns boolean / categorical decisions the downstream workflow can act on.

CHAPTER 04

Architecture overview

flydocs is two cleanly separated runtimes glued by dependency injection. *fireflyframework-pyfly* owns the service runtime — boot lifecycle, dependency injection, command-query responsibility segregation, REST controllers, event-driven architecture, observability, security, actuator endpoints. *fireflyframework-agentic* owns the AI runtime — multimodal LLM agents (*FireflyAgent*), the prompt registry, and the *PipelineEngine* DAG runtime that powers the orchestrator.

Layered layout

Layer	Responsibility	Constraint
interfaces/	Public DTOs and enums — the stable HTTP contract.	The only thing the public REST API speaks. SQLAlchemy and framework types must not leak into responses.
models/	SQLAlchemy entities and async repositories. ExtractionJob is the main entity.	Repository methods are async; the service stays async-first end to end.
core/services/	Pipeline stages, validators, orchestrator, workers, webhook publisher, transformation engine.	Pure async Python. No imports from web/. Each stage is an isolated, testable service.
core/configuration.py	IDPCoreConfiguration — the single @configuration that declares every cross-cutting @bean.	The single place outside the stereotype decorators where beans live. All wiring is auditable from one file.
web/controllers/	@rest_controller classes that adapt HTTP to the CQRS bus. Body validation, RFC 7807 errors, idempotency headers.	Pure FastAPI adapters. No domain logic. Handlers are the unit of work; the bus is the seam for cross-cutting concerns.
web/advice/	@controller_advice that maps domain exceptions to RFC 7807 problem responses.	One class, one place. Domain code throws semantic exceptions; the advice translates them to HTTP.
resources/prompts/	YAML prompt templates (one per LLM stage). Loaded at boot by PromptCatalog and registered with the framework-wide PromptRegistry.	Every LLM stage's prompt is data, not code. Edits ship as YAML diffs.

Boot sequence

The boot sequence is short and deterministic. Uvicorn loads the FastAPI app from flydocs.main, which builds a PyFlyApplication. The lifespan hook runs pyfly_app.startup(), which scans the configured packages, instantiates every @configuration, resolves the bean graph, mounts every @rest_controller on FastAPI, starts the actuator endpoints, and turns on the observability stack (structlog JSON, OpenTelemetry, Prometheus). By the time the first request lands, every cross-cutting service the controller depends on is warm.

Dependency injection — four mechanisms

fireflyframework-pyfly exposes four complementary ways to put a class in the container. flydocs uses all four — the choice is significant.

Mechanism	Used for	Example
@configuration + @bean	Anything not picked up by a stereotype decorator: repositories, the prompt catalog, the LLM stages, the orchestrator, health indicators.	core/configuration.py::IDPConfiguration
@service	Domain classes that should be scanned and registered automatically. The constructor's type annotations drive injection.	BinaryNormalizer, PdfGuard, ImageNormalizer, ...
@command_handler / @query_handler	Handlers dispatched by the CommandBus / QueryBus. Frozen dataclasses parameterised by their return type drive routing through Generic introspection.	ExtractHandler[ExtractCommand, ExtractionResult]
@rest_controller	Controllers mounted on FastAPI. Routes are declared with @get_mapping / @post_mapping; bodies validated through Valid[Body[...]] with RFC 7807 on failure.	ExtractController, JobsController, VersionController

CQRS — controllers stay thin

Controllers never talk to handlers directly. They route every call through the CommandBus / QueryBus, which dispatch by Generic type introspection. Commands and queries are frozen dataclasses parameterised by their return type. The handler is the unit of work; the bus is a clean seam where cross-cutting concerns (tracing, metrics, future retries) plug in without touching the controller or the handler.

```
@command_handler
@Service
class ExtractHandler(CommandHandler[ExtractCommand, ExtractionResult]):
    def __init__(self, orchestrator: PipelineOrchestrator,
                 settings: IDPSettings) -> None:
        self._orchestrator = orchestrator
        self._settings = settings

    async def do_handle(self, command: ExtractCommand) -> ExtractionResult:
        timeout = self._settings.sync_timeout_s
        return await asyncio.wait_for(
            self._orchestrator.execute(command.request),
            timeout=timeout,
        )
```

Layout invariants

Six intentional constraints keep the architecture from drifting under maintenance pressure. Each one earns its keep by catching a real category of bug at code-review time or at boot.

Invariant	Reason
interfaces/ is the only module the public HTTP API speaks.	Prevents SQLAlchemy / framework types from leaking into responses. The contract stays stable across internal refactors.
core/services/ is pure async Python; no imports from web/.	Keeps the domain layer testable without spinning up FastAPI. Stages can be re-used by the worker and by future surfaces (gRPC, MCP, CLI).

Invariant	Reason
web/ is @rest_controller beans only.	Controllers and services don't depend on FastAPI either — a future migration off FastAPI is a per-controller rewrite, not a domain rewrite.
core/configuration.py is the single place where extra @beans live.	All wiring in one file. The bean graph is auditable from one read.
Commands and queries are frozen dataclasses.	Pyfly introspects Generic args for bus routing; frozen makes equality / hashing free.
Document bytes never hit the database.	extraction_jobs.schema_json holds base64 only because the worker re-renders the request. Replace with a blob-store pointer if your DB can't keep this.

CHAPTER 05

The pipeline

The orchestrator is the only entry point from the CQRS layer into the LLM pipeline. It is a plain *fireflyframework-pyfly* bean that exposes a single async method, `execute(request)` → `ExtractionResult`. Every call builds a fresh *fireflyframework-agentic* `PipelineEngine` DAG from the toggles in `request.options.stages`, runs it, and assembles the result. The trace and the event log reflect exactly what executed — a request that disables judge produces a DAG without a judge node.

Stages at a glance

#	Stage	Mandatory?	What it does
1	load	yes	Sniff media type and count pages on every input file. Pure Python. No LLM call.
2	discover	no	LLM enumerates every sub-document inside each unpinned, multi-page file. One segment per sub-document with a page range.
3	classify	no	LLM assigns each segment a declared DocSpec — or marks it unmatched. Per-segment fan-out via <code>asyncio.gather</code> .
4	plan_tasks	yes	Pure Python: build the flat (segment, DocSpec) task list every downstream stage iterates.
5	extract	yes	LLM produces fields + normalised bboxes for every task. Fanned out with <code>asyncio.gather</code> .
6	bbox_validation	yes	Pure-Python geometric hallucination check: stamps every bbox with a quality verdict and continuous score.
7	bbox_refine	no	Replaces LLM coordinates with grounded ones via the hybrid matcher (rapidfuzz first, LLM only for residual). Pluggable OCR engine.
8	field_validation	no	Pure-Python validation: regex / enum / range + every <code>StandardValidator</code> declared per field.
9	visual_authenticity	no	LLM evaluates caller-defined visual checks (signature, stamp, photo, MRZ, ...).
10	content_authenticity	no	LLM audit of document coherence: dates consistent, totals tally, boilerplate present, tampering signals.
11	judge	no	Second LLM pass re-grades every extracted value against the source.
12	judge_escalation	no	When the judge's failure rate exceeds the threshold, re-run extract + judge with a stronger model.
13	transform	no	Caller-declared post-extraction transformations: declarative entity resolution + free-form LLM transforms.
14	rules	no	LLM evaluates the business-rule DAG level by level.
15	assemble	yes	Pure Python: compose the <code>ExtractionResult</code> .

How the DAG is built per request

The orchestrator does not hardcode the pipeline. It builds a fresh DAG per request, choosing the nodes from `StageToggles` on `ExtractionOptions`. Three properties fall out of this design.

Property	Why it matters
The trace mirrors the request.	If the caller disables judge, the DAG has no judge node and the event log shows the exact path executed. Audits and post-mortems are reproducible from the trace alone.
Per-stage timeouts.	Each <code>CallableStep</code> carries its own <code>timeout_seconds</code> . A slow LLM call cannot drag the whole request beyond its budget — only its stage cancels.

Property	Why it matters
Failure isolation.	A CallableStep that raises is captured by the engine. The orchestrator records the failure in pipeline_errors and the request still returns a partial result. The request never fails for a single stage error.

Concurrency model

Concurrency lives in three places. Per-document fan-out inside a stage runs through `asyncio.gather` — one slow document blocks only its siblings transiently; the stage finishes when the slowest one does. Per-stage timeouts cancel individual stages without affecting the rest of the DAG. And the outer per-request wall is `FLYDOCS_ASYNC_TIMEOUT_S` (1800 s default) for the queued path and `FLYDOCS_SYNC_TIMEOUT_S` (60 s default) for the inline path.

Failure isolation

A stage that raises does not abort the request. The orchestrator catches the exception, appends a structured entry to `pipeline_errors[]`, and continues. Downstream stages still run with whatever state is available. This is how the service returns partial results: the field extraction succeeded, only the judge bailed. The caller can decide whether to accept the partial result or re-submit with the failed stage disabled.

```
// Excerpt of a partial-success response
{
  "documents": [{ /* extracted fields are present */ }],
  "pipeline_errors": [
    { "node": "judge",
      "code": "JUDGE_ERROR",
      "message": "llm provider timed out after 180s" }
  ]
}
```

Bbox refinement, sync vs. async

The synchronous surface runs `bbox_refine` inline. The asynchronous surface skips it inline — `JobWorker` mutates `stages.bbox_refine = False` before calling the orchestrator, then publishes `IDPBboxRefineRequested` on success. A dedicated `BboxRefineWorker` grounds bounding boxes out of band; the job transitions through `SUCCEEDED` → `PARTIAL_SUCCEEDED` → `SUCCEEDED` (or `REFINING_BBOXES` → `SUCCEEDED`). The refiner is idempotent — fields with `bbox.source ∈ {pdf_text, ocr}` are skipped on re-run.

CHAPTER 06

Binary normalization

The first thing flydocs does with any caller-supplied binary is turn it into one or more LLM-renderable rows. The BinaryNormalizer service decides — by magic-byte sniffing first, then by the declared content type — how the input should be handled, and emits a list of NormalisedBinary rows that downstream stages can ship straight to the multimodal LLM. A single PDF becomes one row; a ZIP becomes one row per archive member; an EML with attachments fans out into one row per attachment.

Input class	Routing	Outcome
PDF	PdfGuard integrity check, passthrough.	Sent as application/pdf with its real page count.
PNG / JPEG / GIF / WebP	Native to every supported LLM provider. Passthrough.	Sent as the source media type.
HEIC / HEIF / AVIF / multi-frame TIFF / BMP / SVG	ImageNormalizer (Pillow + pillow-heif + cairosvg).	Re-encoded into PNG or multi-page PDF the providers can read.
DOC / DOCX / XLS / XLSX / PPT / PPTX / ODT / ODS / ODP / RTF / HTML	OfficeConverter — Gotenberg sidecar by default; LibreOffice fallback for slim/dev images.	Converted to PDF and routed through the PDF lane.
ZIP / 7z / TAR / GZIP	ArchiveUnpacker, then recurse on each member.	Fan-out: one row per archive member, with derived_from chaining back to the archive name.
EML / MSG	EmailUnpacker. Body and attachments each route through the normalizer.	Fan-out across the email body and every attachment.
Plain text / Markdown / CSV	Wrapped in a synthetic ... HTML envelope, routed through the office converter.	Rendered as a PDF the LLM can read without pulling pandoc into the runtime.
Encrypted / corrupt PDF	Typed BinaryNormalizationError.	Mapped to RFC 7807 422 by the exception advice — caller fixes the input rather than getting a silent failure.

Recursion depth and total fan-out are bounded by FLYDOCS_BINARY_MAX_RECURSION_DEPTH and FLYDOCS_BINARY_MAX_EXPANDED_FILES — a malicious zip bomb cannot expand into an unbounded number of LLM calls. Office conversion is pluggable behind the OfficeConverter protocol; the default GotenbergConverter POSTs to a Gotenberg sidecar so the runtime container can stay distroless, and LibreOfficeConverter shells out to soffice as a fallback for slim/dev images.

Operator note

Office and image conversion are pluggable. Adding OnlyOfficeConverter, CollaboraConverter, or a managed cloud Office service is a new bean plus a new value on the FLYDOCS_OFFICE_CONVERTER setting. The protocol stays the same; the rest of the pipeline does not change.

CHAPTER 07

Extraction and bounding-box grounding

The extractor is the only LLM stage that is always on. It is also the most expensive call in a typical request, so the service is engineered to keep its inputs lean and its outputs verifiable.

The multimodal extractor

For every (segment, DocSpec) task, the extractor builds a dynamic Pydantic model from the field schema, renders the flydocs/extract prompt template with the schema JSON, the media type, the page count, the caller's intention, and an optional language hint, then composes the user-side multimodal content list as [user_text, anchor?, BinaryContent]. The agent runs through FireflyAgent.run(content); the output is post-processed to clamp every bbox to [0, 1], coerce types, and populate pagesFound.

Failure is graceful. If the primary model fails (timeout, content policy, unsupported variant) and FLYDOCS_FALLBACK_MODEL is set to a different model id, the extractor retries on the fallback. The actual model used per task is reflected in result.model; when multiple models contributed, they are joined with a comma so the caller can audit exactly which engine produced which value.

Optional pre-extraction text anchor

When FLYDOCS_EXTRACTION_TEXT_ANCHOR=docling is set, the extractor splices a Markdown rendering of the document — produced by IBM Docling — between the instruction text and the binary content. The LLM sees a cleaned textual view of the page right next to the actual image. This is measurably helpful on multilingual scans where the vision model misreads diacritics, on long PDFs where attention truncates, and on dense tabular documents where row/column structure is hard to follow visually. The anchor is best-effort: a missing dependency raises at first use, but a runtime conversion error logs a warning and falls back to binary-only behaviour.

Geometric bbox validation

After every extract, the bbox validator runs on every field — no toggle. It stamps each bounding box with one of good / poor / suspicious / invalid / empty and a continuous quality score in [0, 1]. Heuristics, in priority order, catch the classic LLM hallucination of a generic full-page region:

Verdict	Trigger	Default score
empty	fieldValueFound is None or zero-area BoundingBox.empty().	0.0
invalid	Corners outside [0, 1], xmin ≥ xmax, or ymin ≥ ymax.	0.0
suspicious	Area > 0.7 of the page, or covers > 0.9 horizontally / vertically.	0.2
poor	Area < 5e-5 (≈ 5px × 5px on a 1000px-wide render) or extreme aspect ratio (height/width > 30 or < 1/30).	0.4
good	Anything else. Score combines area, aspect, and margin sanity.	0.5 – 1.0

Grounded bbox refinement

LLM-estimated bounding boxes are imprecise — they land in roughly the right region of the page but routinely miss the actual text by one or more lines. The geometric validator cannot catch this. The `bbox_refine` stage fixes it by grounding every value against the document's real text layer:

Source	Adapter	When it applies
pdf_text	PyMuPDF — sub-pixel-accurate word coordinates via <code>page.get_text("words")</code> .	Born-digital PDFs with a real text layer.
ocr (tesseract)	Pytesseract — default OCR engine, no PyTorch.	Image-PDFs and rasters when <code>FLYDOCS_BBOX_REFINE_OCR_ENGINE=tesseract</code> .
ocr (docling)	IBM Docling — Heron layout model + RapidOCR. Detects tables and reading order; tie-breaks for the value matcher.	Image-PDFs and rasters when <code>FLYDOCS_BBOX_REFINE_OCR_ENGINE=docling</code> .
llm (kept as-is)	When the matcher cannot locate the value above its threshold, the LLM bbox is kept, tagged <code>source=llm</code> , <code>refinement_confidence=null</code> .	Documents where neither text layer nor OCR resolves the value.
none (no refinement)	NoneOcrEngine — image pages keep the LLM bbox; born-digital PDFs still go through PyMuPDF.	Lightweight deployments where OCR cost is not justified.

Hybrid value matcher

The default matcher is a cascade: rapidfuzz first, LLM only for the residual that fuzzy matching cannot resolve. The combination grounds 70-90% of fields without an LLM call while still handling spelled-out numbers, date format variants, and multilingual quirks via the LLM fallback. The matcher can be flipped to fuzzy-only for deterministic, zero-LLM grounding, or to llm-only for environments where rapidfuzz is undesirable.

CHAPTER 08

Validation, authenticity, and the judge

Three layers turn a raw multimodal extraction into something an enterprise back office can trust. They are independent, composable, and individually optional — but together they answer almost every question a compliance team would ask before accepting an automated decision.

Deterministic field validators

Pure-Python checkers that run after extraction, never call the LLM, never touch the network, and add a few milliseconds of latency. The registry is open: dropping in a new checker is a single function plus a registry entry plus a unit test. The library ships with thirty-plus validators across seven families.

Family	Validators
Network	email, uri, url, domain, slug, ipv4, ipv6.
Temporal	date, datetime, time, iso_8601.
Identifiers	uuid, json, hex_color.
Finance	iban (mod-97), bic, credit_card (Luhn), currency_code (ISO 4217), amount (allow_zero / allow_negative).
Telephony	phone_e164 (E.164 shape after normalisation).
Geographic	country_code (ISO 3166-1), language_code (ISO 639-1), postal_code (country-aware: ES, FR, DE, IT, GB, US, PT, NL, BE, BR), latitude, longitude.
National identifiers	nif (ES, mod-23), nie, cif, vat_id (EU, per-country), ssn (US), passport_number.

Every validator instance carries a severity. The default error flips the field to invalid. warning records the finding but keeps the field valid — useful when two checkers race (a value that is either a Spanish NIF or a NIE, declared with both as warnings, will always have exactly one warning fire without flipping the field).

Visual authenticity

One LLM call per document. The caller declares a list of yes/no visual checks (each a name + description); the model looks at the document and returns a VisualValidationOutcome with passed, confidence, and notes for every check. Typical checks: signature present, notary stamp present, MRZ band present, photo present, watermark intact, security ribbon visible.

Content authenticity

One LLM call per document. No caller-defined checks — the model picks the coherence audits itself, guided by the document type, description, country, and intention. Output: a ContentAuthenticity aggregate with an overall integrity status (VALID / INVALID / UNCERTAIN) and a list of named checks each carrying status, evidence, and reasoning. This is the layer that catches problems like dates that don't agree, totals that don't add up, expected boilerplate that isn't there, and visual cues of tampering.

The judge

One LLM call per document. The judge sees the document bytes and the JSON dump of the extraction result. For every field, it returns a status (PASS / FAIL / UNCERTAIN), a confidence score, evidence (the exact quote or region it matched), notes, and a `flag_for_review` bit. The service mutates the existing `ExtractedFields` in place by populating their judge attribute. The judge is strict by design: a value that is plausible but not supported by the document gets FAIL. This is the cheapest defence against multimodal hallucinations.

Judge escalation

Production deployments typically run a cheap primary model (claude-haiku, gpt-4o-mini) and a stronger escalation model (claude-opus, gpt-4o) used only when the judge flags too many fields. flydocs automates that policy. When the judge's failure rate exceeds the `escalation_threshold`, the orchestrator re-runs `extract` + `judge` with the escalation model and keeps the better result. The accepted-or-rejected outcome lands on `result.escalation`, an audit block that records the primary model, the escalation model, both failure rates, and whether the new fields replaced the originals.

Cost pattern

The cheap-by-default, escalate-on-uncertainty pattern lets operators amortise LLM spend. Typical production traffic splits 80/20: 80% of requests clear the cheap model; 20% trigger escalation. Token spend is dominated by the cheap lane; accuracy stays near the strong-model ceiling on the documents that actually need it.

CHAPTER 09

Post-extraction transformations

The transform stage applies caller-declared transformations to the extracted field groups *after* every other LLM stage (extract, judge, judge_escalation) and *before* rules and assemble. It moves deduplication, normalisation, role classification, language translation, and schema migration into the IDP itself — rather than having every consumer rebuild them.

Type	Cost	Right use cases
entity_resolution	Free, milliseconds.	Deduplicate rows that refer to the same entity. Bridges accent variants, partial names, and formatting differences in identifiers across documents.
llm	One LLM call per group.	Anything the declarative types cannot express: role buckets, summarisation, free-text normalisation, anonymisation, schema migration, multilingual normalisation.

Declarative entity resolution

A deterministic two-phase matcher. Phase one merges rows whose normalised value of a given identifier field (dni, cif, ...) collide; the normaliser strips formatting so 07.549.861-L and 07549861L resolve to the same key. Phase two falls back to NFKD-fold + token-subset matching for rows without an identifier; two rows match when one name's token set is a subset of the other AND they share at least min_shared_tokens tokens (default 2, which bridges accent and partial-name variants without collapsing strangers who share a single first name). Canonical-row selection picks the most complete value per sub-field — longest string wins for names; first non-empty wins for other types.

Scope: per-task vs. per-request

Each transformation declares a scope. The default task scope runs once per (segment, DocSpec) task and mutates that task's groups in place — right for single-document transformations. The request scope concatenates the matching target_group across every task in the request, applies the transformation once over the consolidated rows, and emits the result as a new entry under result.request_transformations — right for cross-document entity resolution, where the same person mentioned in five deeds collapses into a single canonical row.

Free-form LLM transformations

The escape hatch. The caller supplies a one-sentence intention; the engine renders a focused prompt against the target group's rows and expects the LLM to return rows in the same shape. The response replaces (or, with output_group, augments) the original group. Common use cases: collapse free-text cargo values into a closed taxonomy ({administrador_unico, consejero, apoderado, otros}), translate every value to English while preserving keys, replace each name with a stable pseudonym for anonymisation, collapse the list into one summary row per distinct identifier.

Transformations are applied in declared order, so callers can chain them: entity_resolution first (cheap, deterministic), then an llm step that operates on the deduped survivors. Failures of individual transformations are caught by the engine; the surrounding pipeline never fails because one transformation misbehaved.

CHAPTER 10

The business rule engine

Rules turn extracted data into *decisions* the surrounding workflow can act on. *Should we auto-approve? Send to manual review? Reject? Escalate to fraud?* These are the questions a hand-coded if/else block ends up answering in every IDP deployment. flydocs lets callers declare them as data and resolve them with an LLM evaluator that walks their dependency graph.

Anatomy of a rule

```
{
  "id": "kyc_complete",
  "predicate": "All identity fields are populated AND nif is valid.",
  "parents": [
    {"parentType": "field", "documentType": "passport",
     "fieldNames": ["full_name", "nif"]},
    {"parentType": "validator", "documentType": "passport",
     "validatorName": "photo_present"}
  ],
  "output": {
    "type": "boolean",
    "valid_outputs": ["true", "false"]
  }
}
```

Field	Meaning
id	Stable identifier. Other rules reference it via RuleRuleParent.ruleId.
predicate	Natural-language decision statement. The LLM evaluates it.
parents	Declared dependencies. Three kinds: field (fields the rule reads), validator (a visual-validator outcome the rule reads), rule (another rule's id; creates the DAG edge).
output.type	boolean, string, or number. Drives the expected output shape.
output.valid_outputs	Optional closed set. If the LLM returns something outside, the rule is flagged for human review.

DAG evaluation

The engine walks the rule graph with the standard library's `graphlib.TopologicalSorter`. Every level is *one* LLM call that evaluates all rules whose parents are resolved. The token bill and the latency therefore scale with *rule depth* rather than rule count. Cycles are rejected by `TopologicalSorter.prepare()` before any LLM call fires — a regression here would surface immediately at request validation rather than as an infinite loop in front of the model.

Scoped context

The prompt is scoped: only the fields, validator outcomes, and prior rule outputs that a rule declares as parents are included in its context. The engine does not dump the whole extraction into every rule's prompt. This keeps the rule engine cheap in token terms and forces a clean, auditable dependency declaration in the rule definitions.

Worked example — Spanish power-of-attorney deed

Three rules — KYC completeness, distinct-parties check, and a recency check — all sit at level one, with no inter-rule dependencies. The engine resolves them in one LLM call.

Rule	Predicate (natural language)	Result on a self-power
kyc_complete	Both otorgante_nombre and apoderado_nombre are populated, and both DNI/NIE values are populated, and fecha is populated.	true — all five required KYC fields are present.
parties_distinct	otorgante_nombre and apoderado_nombre refer to different individuals (case- and accent-insensitive).	false — the otorgante and the apoderado are the same person.
recent_document	fecha is on or after 2020-01-01.	true — document date 2025-05-15 is after 2020-01-01.

The parties_distinct: false result is the rule engine catching something a hand-coded extractor would have missed: the deed is a self-apoderamiento where the same DNI appears on both sides. The natural-language predicate makes this kind of semantic comparison cheap to express; the LLM evaluator makes it cheap to run.

Design tip

Use rules for semantic judgements that benefit from LLM flexibility — distinct-parties, age-bracket bucketing, category mapping, KYC completeness. Use deterministic validators for arithmetic checks ("line items sum to total") and shape checks (IBAN, NIF, ISO date). Each tool plays to its strengths.

CHAPTER 11

Asynchronous jobs and event-driven backbone

The asynchronous surface is the same pipeline, the same request shape, and the same response — wrapped in a durable event-driven backbone. The caller submits a job via `POST /api/v1/jobs`, receives a 202 with a job id, and either polls `GET /api/v1/jobs/{id}` for status, long-polls `GET /api/v1/jobs/{id}/result` for the final result, or waits for the HMAC-signed webhook to land on the callback URL.

Topology

Component	Role
API	One or more unicorn workers behind a load balancer. Stateless; no sticky sessions. Publishes <code>IDPJobSubmitted</code> events on the EDA bus.
Worker	One or more processes that subscribe to the EDA bus via <code>fireflyframework-pyfly's EventPublisher.subscribe</code> . Each event is delivered to exactly one consumer in the <code>flydocs-workers</code> consumer group.
BboxRefineWorker	Dedicated worker for out-of-band bbox grounding. Listens for <code>IDPBboxRefineRequested</code> and transitions jobs from <code>PARTIAL_SUCCEEDED</code> to <code>SUCCEEDED</code> once grounded coordinates are written.
Postgres	Holds <code>extraction_jobs</code> and the EDA outbox (<code>pyfly_eda_outbox</code> + <code>pyfly_eda_offsets</code>). With the default Postgres EDA adapter there is no separate broker.

Pluggable broker

The default postgres EDA adapter uses a durable outbox plus `LISTEN/NOTIFY` in the same database the service already owns — no extra broker, no extra failure mode. The adapter is pluggable: flip `FLYDOCS_EDA_ADAPTER` to memory (single-process dev), redis (Redis Streams), or kafka (aiokafka). The orchestrator and the worker do not care; only the bus implementation changes.

Idempotency

Submissions honour an `Idempotency-Key` header. When the same key is seen twice, the controller replays the original `SubmitJobResponse` rather than creating a duplicate job. The key plus the request hash become the primary key in `extraction_jobs`; retries from a flaky client never multiply load.

Retry policy

Failures are classified into two buckets. Permanent errors (content policy, invalid API key, unsupported model, validator errors) skip the retry budget — retrying will not help, and the caller needs to fix the input. Retryable errors (timeouts, network errors, transient 5xx, generic `RuntimeError`) are re-queued with exponential backoff ($\min(\text{retry_max_delay_s}, \text{retry_base_delay_s} * 2^{(\text{attempt}-1)})$ plus 20% jitter). The attempts counter is persisted atomically in Postgres so the budget survives worker restarts. When attempts reaches `FLYDOCS_JOB_MAX_ATTEMPTS`, the job goes to `FAILED` even if the last error was retryable.

HMAC-signed webhooks

On every terminal job state, the WebhookPublisher posts the full ExtractionResult to the caller's callback_url. The payload mirrors the EDA event envelope (event_id, event_type, occurred_at, correlation_id, tenant_id) and is signed with HMAC-SHA256 in the X-Flydocs-Signature header using FLYDOCS_WEBHOOK_HMAC_SECRET. Deliveries retry on 5xx and 429 up to FLYDOCS_WEBHOOK_MAX_ATTEMPTS (default 5); 4xx is treated as permanent.

Security note

Webhook HMAC verification is mandatory in production. Set FLYDOCS_WEBHOOK_HMAC_SECRET to a strong random string; validate the X-Flydocs-Signature header on every callback in the receiver. The publisher signs the exact byte-for-byte payload, so verification is a constant-time HMAC compare on the raw body.

CHAPTER 12

Observability, cost telemetry, and prompt caching

flydocs treats observability as a first-class concern. Every request carries a correlation id from the inbound HTTP header through every pipeline stage, every external call, every outbound webhook, and every database row. The service emits three telemetry streams that share that correlation id, so any investigation is a grep, not a join.

Three telemetry streams

Stream	Surface	What it carries
Metrics	Prometheus at /actuator/metrics.	CQRS handler latency histograms, HTTP latency, runtime metrics. Default labels include the bus adapter and the pipeline stage.
Traces	OpenTelemetry — configure with standard OTEL_EXPORTER_OTLP_* env vars.	One span per pipeline node, one span per outbound LLM / webhook call. W3C trace context is propagated to every downstream HTTP call.
Logs	structlog JSON to stdout.	Every line carries request_id. node_start, node_done, node_failed, and per-stage diagnostics are emitted as structured events.
Outbound-call log	Single log-line-oriented record per external call.	outbound_call target=... op=... status=... latency_ms=... model=... in_tokens=... out_tokens=... cost_usd=... — everything ops needs for spend forensics in one place.

Cost telemetry

Every FireflyAgent.run records a UsageRecord with tokens, USD cost, latency, model, agent name, and the correlation id. The orchestrator binds the request id to a context variable at the start of execute(), so every record carries the right correlation id. When the engine returns its PipelineResult, the usage figures are already aggregated for the request; the orchestrator maps them into the response's usage field — total tokens, total cost, breakdown by agent (extractor, classifier, splitter, judge, visual, content, rule-engine), and breakdown by model.

Pricing data is sourced from genai-prices, a provider-agnostic live tariff catalogue. Anthropic, OpenAI, Google, Mistral, and any other provider *fireflyframework-genai* knows about are priced uniformly. The full Claude 4 family (opus-4-*, sonnet-4-*, haiku-4-*) is covered out of the box. Cost telemetry works without any local override.

Provider-aware prompt caching

Every FireflyAgent ships with a shared PromptCacheMiddleware. The middleware detects the active provider via `fireflyframework_agentic.model_utils.detect_model_family` and writes the right cache hints into pydantic-ai's `model_settings` — Anthropic and Bedrock-Anthropic get explicit `cache_control` breakpoints on the system prompt and the last user-message block; OpenAI and Azure-OpenAI get an `openai_prompt_cache_key` for automatic caching above 1024 tokens; Google Gemini accepts caller-supplied `CachedContent` resource ids.

Provider	Cache hint	Effect
anthropic:claude-* / bedrock:anthropic.claude-*	Explicit <code>cache_control</code> on instructions + last user block. 5-minute or 1-hour TTL.	~90% cost reduction on cached input.
openai:gpt-* / azure:gpt-*	Automatic caching above 1024 tokens. Stable <code>openai_prompt_cache_key = ffa-{agent_name}</code> for cache-backend affinity.	~50% cost reduction on cached input.
google:gemini-*	Pre-created <code>CachedContent</code> resource ids passed through to pydantic-ai. Caller owns the resource lifecycle.	~25% cost reduction on the cached portion.
mistral:* / cohere:* / deepseek:* / unknown	No cache primitive currently exposed via pydantic-ai. Debug log only.	No discount.

Cache writes and reads surface as `cache_write` / `cache_read` on the `outbound_call` log lines; aggregated values land on `usage.cache_creation_tokens` / `usage.cache_read_tokens` in the response. The whole middleware can be disabled with `FLYDOCS_PROMPT_CACHE=off` for A/B benchmarking or for one-shot workloads where the per-call write premium exceeds the read discount.

W3C trace context

`traceparent`, `tracestate`, `X-Correlation-Id`, `X-Request-Id`, and `X-Tenant-Id` propagation are handled by `pyfly.observability.correlation` and the default `CorrelationFilter` in the web filter chain. There is no local middleware to wire. Every response echoes the headers back; every outbound HTTP request carries them forward.

CHAPTER 13

Deployment and security

flydocs is designed to drop into an existing Kubernetes or container deployment with no bespoke infrastructure. The default posture is a Postgres-only data plane — the service already runs Postgres for persistence, so reusing it as the EDA outbox removes one operational dependency.

Topology

A production deployment has three moving parts: the API (uvicorn behind a load balancer), one or more workers, and a Postgres instance that holds `extraction_jobs` and the EDA outbox (`pyfly_eda_outbox` + `pyfly_eda_offsets`). The API and the workers share the same container image; the difference is which CLI subcommand starts them (flydocs serve vs. flydocs worker). Redis Streams and Kafka are still supported as alternative EDA backends — the choice is an environment variable flip.

Image variants

Tag prefix	Architectures	What's inside	Pull when
(none) — latest, vX.Y.Z, main	linux/amd64 + linux/arm64	Slim runtime: PyMuPDF + Tesseract OCR. No PyTorch. Distroless-friendly.	Default. Use unless you need layout-aware OCR or the Markdown text-anchor.
docling- (docling-latest, docling-vX.Y.Z)	linux/amd64 only	Slim image plus the docling extra: PyTorch + Hugging Face model loaders (~2.5 GB). Unlocks the Heron layout model and the Markdown text-anchor.	When you want layout-aware grounding on noisy scans or Markdown anchoring on dense tabular / multilingual documents.

Health and readiness probes

`/actuator/health/readiness` aggregates two health indicators: `database_health` (SqlAlchemyHealthIndicator pings the async engine with `SELECT 1`) and `eda_health` (EventPublisherHealthIndicator reports the active adapter — `PostgresEventBus`, `RedisStreamsEventBus`, `KafkaEventBus`, or `InMemoryEventBus`). When any indicator is DOWN, the endpoint returns 503 and the load balancer or kubelet stops routing traffic.

Security posture

Concern	Posture
Document bytes at rest	Never written to the service-side filesystem. Only the base64 payload sits in extraction_jobs.schema_json for the job lifetime. Replace with a blob-store pointer if your DB cannot hold this.
Webhook authentication	Mandatory HMAC-SHA256 in production via FLYDOCS_WEBHOOK_HMAC_SECRET. The publisher signs every payload byte-for-byte.
API authentication	Entry-level gate via FLYDOCS_API_KEYS (comma-separated). For OIDC / OAuth2, swap in fireflyframework-pyfly's security-jwt starter and register a JWT decoder bean.
LLM credentials	Provider credentials are read from standard env vars (ANTHROPIC_API_KEY, OPENAI_API_KEY, GOOGLE_API_KEY, MISTRAL_API_KEY). fireflyframework-genai resolves the right one from the model id prefix. Use your secrets manager; never bake them into the image.
Multi-tenancy	X-Tenant-Id is propagated end to end and recorded on every EDA event / webhook envelope. Per-tenant quotas and routing can be added at the gateway.
TLS	Terminated at the load balancer. The service serves plain HTTP inside the cluster, in line with sidecar / service-mesh conventions.
PII boundaries	Pipeline errors and outbound_call log lines carry stage codes and model ids, not field contents. The judge's evidence quotes are deliberate by design and live in the response, not in logs.

CHAPTER 14

Extensibility and the Firefly Framework

Every flydocs subsystem is engineered to extend without touching unrelated code. The combination of dependency injection, protocol-based plug-points, and a YAML-backed prompt catalog turns most extensions into a new class plus a one-line registry entry.

Want to...	Touch points
Add a new standard validator	Append a member to <code>interfaces/enums/standard_validator.py::StandardValidatorType</code> ; implement <code>_check_</code> in <code>core/services/validation/standard_validator_registry.py</code> and register it in CHECKERS; add a unit test.
Add a new pipeline stage	Build a service under <code>core/services/</code> . Add a <code>@bean</code> in <code>IDPCoreConfiguration</code> . Wire it into <code>PipelineOrchestrator</code> . Gate it behind a new <code>StageToggles</code> flag (default <code>False</code>). Bundle the prompt as YAML.
Add a new transformation type	Extend the discriminated union in <code>interfaces/dtos/transformation.py</code> with a new <code>_BaseTransformation</code> subclass and a unique literal discriminator. Add a branch in <code>TransformationEngine._dispatch</code> . No orchestrator or public-API changes.
Add a new OCR engine	Implement the <code>OcrEngine</code> protocol. Add a new value to <code>FLYDOCS_BBOX_REFINE_OCR_ENGINE</code> . Add a bean branch in <code>IDPCoreConfiguration</code> . The rest of the <code>bbox-refine</code> flow is unchanged.
Add a new Office converter	Implement <code>OfficeConverter</code> (e.g. <code>OnlyOfficeConverter</code> , <code>CollaboraConverter</code>). Add a bean branch. Add the env value.
Add a new text anchor	Implement the <code>TextAnchor</code> protocol. The extractor's <code>_build_user_content</code> already routes any <code>TextAnchor</code> that returns a non-empty Markdown render into the user message ahead of the binary.
Add a new EDA broker	Add a new adapter to <code>pyfly.eda.auto_configuration</code> and flip <code>FLYDOCS_EDA_ADAPTER</code> . The orchestrator and the worker do not need to know.
Edit a prompt	Bump the version in the YAML, edit the template, run the real-LLM smoke test. The catalog reloads at process start.

The Firefly Framework dividend

Every cross-cutting concern in flydocs is delegated to the framework. The business logic stays small; the framework handles the things that would otherwise leak across modules.

Concern	Provided by
Dependency injection	<code>fireflyframework-pyfly @configuration + @bean</code>
CQRS — commands, queries, bus	<code>fireflyframework-pyfly @command_handler / @query_handler</code>
REST surface	<code>fireflyframework-pyfly @rest_controller</code> over FastAPI
Async pipeline DAG	<code>fireflyframework-agentic PipelineEngine / PipelineBuilder</code>
Prompt management	<code>fireflyframework-agentic PromptTemplate + PromptRegistry</code> (YAML-backed)
Multimodal LLM agents	<code>fireflyframework-agentic FireflyAgent</code> over <code>pydantic-ai</code>

Concern	Provided by
EDA — events, durable outbox, retries	fireflyframework-pyfly EventPublisher (Postgres / Redis / Kafka / in-memory)
W3C trace context propagation	fireflyframework-pyfly CorrelationFilter + pyfly.observability.correlation
Health, metrics, actuator	fireflyframework-pyfly @enable_core_stack — structlog JSON, OTLP traces, Prometheus, /actuator/*
Persistence	fireflyframework-pyfly data-relational starter + SQLAlchemy async + Alembic
RFC 7807 error responses	@controller_advice exception advice mapping domain errors to problem+json

CHAPTER 15

Use cases

flydocs is product- and vertical-agnostic. The same service powers radically different workflows because the field schemas, validators, and business rules are caller-defined. Five representative deployments illustrate the breadth.

KYC — Know Your Customer

Identity verification on national IDs, passports, residence permits, and utility bills. Field schema declares `full_name`, `document_number`, `date_of_birth`, `expiry`, `address`. Standard validators check NIF / NIE / passport-shape / postal code by country. Visual validators check photo present, MRZ band present, signature present. Content authenticity catches expired documents and dates that don't agree. Rules decide `kyc_complete` and `needs_manual_review`.

KYB — Know Your Business

Corporate due diligence on notarial deeds, articles of association, bank certificates, beneficial-ownership declarations. The splitter stage handles multi-document filings — a single uploaded PDF can contain a deed, an ID, and a utility bill, each routed independently. Entity resolution dedupes administrators and shareholders across documents. Rules decide directorship validity, self-power detection, and recency.

Invoice and accounts-payable intake

Schema-driven extraction of supplier, invoice number, dates, line items, totals, and tax breakdowns. IBAN, BIC, and currency-code validators run pure-Python. Content authenticity verifies the totals tally and the tax calculations are internally consistent. Rules categorise the invoice for routing — `approve_under_threshold`, `route_to_buyer`, `escalate_no_po`.

Insurance claims triage

First-notice-of-loss bundles arriving as EML with mixed attachments — claim forms, photographs, repair estimates, police reports. The binary normalizer fans them out; each attachment is classified against the declared DocSpecs. Visual authenticity confirms the expected stamps and signatures. Rules decide auto-approval thresholds, fraud-flag escalation, and assignment to a claims handler.

Notarial and legal review

Power-of-attorney deeds, sales contracts, and mortgage instruments. Visual validators check the notary's signature and seal. The judge flags self-power scenarios and parties that do not match the declared role. Rules enforce regulatory constraints (recency, jurisdiction, distinct parties).

The vertical-agnostic dividend

Across all five verticals the codebase changes are zero. The differences live entirely in the caller's JSON: DocSpec field schemas, declared `standard_validators`, declared visual checks, and the rule definitions. A new vertical is a new JSON template, not a new release.

CHAPTER 16

Competitive positioning

The IDP market splits into three categories. Each one addresses a different mix of trade-offs; flydocs sits in a deliberate position relative to each.

Category	Strengths	Limits	vs. flydocs
Hyperscaler Document AI services (AWS Textract, Azure Document Intelligence, Google Document AI)	Managed. Mature OCR. Strong on known forms with vendor-provided extractors.	Per-vertical extractors. New document types require custom training. Limited business-rule layer. Cloud-locked.	flydocs is provider-agnostic and ships its own validation, judge, and rule layers. The same code runs on the customer's choice of LLM provider and infrastructure.
Vertical IDP vendors (invoice, claims, KYC SaaS)	Productised UIs. Pre-built workflows. SLAs.	One vertical at a time. Data leaves the customer's infrastructure. Field schemas locked to the vendor's model.	flydocs runs the customer's own deployment. Field schemas, validators, and rules are caller-defined. The same service serves every vertical.
Build-your-own on a multimodal LLM	Full control. Tight loop with the model provider.	Three quarters of engineering to add bbox grounding, validators, judges, rules, EDA, webhooks, idempotency, cost telemetry, prompt caching, observability.	flydocs ships all of those as opinionated defaults. The customer keeps full control; the disciplines come for free.

What flydocs is not

flydocs is not a workflow engine. It does not own state machines, human-review queues, or case-management tooling. It is a callable service that turns documents into structured decisions; downstream workflow engines (Temporal, Camunda, n8n, custom Step Functions) consume the result.

flydocs is not a fine-tuning platform. It deliberately treats the multimodal LLM as a primitive. Every accuracy gain comes from prompt design, schema design, validator declaration, and the judge / rule layers — not from re-training the model. When a stronger model lands, the deployment upgrades by changing one environment variable.

flydocs is not a Document Management System. It does not store documents long-term; it processes them and forgets them. The base64 payload sits in `extraction_jobs` only for the job's lifetime and can be migrated to an opaque blob-store pointer when the operator prefers.

CHAPTER 17

Roadmap and direction of travel

flydocs ships as v1.0 with the pipeline described in this paper running in production. The roadmap is driven by what early operators have asked for after they reached steady state on the core service.

Theme	Direction
Pluggable OCR adapters	Paddle and Mistral OCR adapters slot into the existing OcrEngine protocol. Tesseract and Docling already ship; Paddle and Mistral land as additional values on FLYDOCS_BBOX_REFINE_OCR_ENGINE.
Table-aware matching	The Docling reading-order metadata already gives the matcher a tie-break signal. The next step is structure-aware matching for array fields whose rows should ground inside the same table row.
Shared Docling converter	A single Docling converter bean shared between DoclingOcrEngine and DoclingTextAnchor, reducing the warm memory footprint when both are enabled.
Granite-docling VLM mode	An alternative grounding mode that uses IBM's Granite-docling VLM on documents where the dedicated VLM noticeably outperforms Heron + RapidOCR.
Caller-side field-level prompt cache	Cache-key strategies for callers that re-process the same expediente against multiple DocSpec variations — the workload pattern where Anthropic prompt caching pays off most.
MCP surface	An MCP server façade so Claude (and any MCP-aware client) can call the extraction service as a tool — useful for human-in-the-loop reviewers using Claude Code or Cowork.
Plugin-marketplace integrations	Cowork / Claude Code plugins that ship pre-bundled DocSpecs and rule packs for common verticals (KYC, KYB, invoice, claims).
Long-term audit storage	Optional pluggable audit-store that captures the full request + result for the retention horizons regulated operators require. Off by default.
Native MCP-based human review	Wire the human-review path through MCP so an analyst tooling agent can pick up flag_for_review cases, inspect the bounding boxes, and resolve them in-line.

Stability commitment

Everything above is purely additive. The public API contract and the StageToggles set are stable; new features land as new toggles, new adapters, or new transformation types. Existing deployments upgrade by pulling a new image and leaving their request payloads untouched.

CHAPTER 18

Economics and operational profile

A useful mental model: flydocs's cost profile is dominated by the multimodal LLM calls, and the operator has fine-grained control over which calls fire on which document.

Three knobs that move the bill

Knob	What it does	Practical impact
Stage toggles	Each non-mandatory stage adds at least one LLM call (per-doc fan-out and per-level rule eval multiply this).	Disabling judge and content_authenticity for high-volume bulk runs halves token spend on those requests without touching extraction accuracy.
Model choice	FLYDOCS_MODEL is the default; options.model overrides per request. Heavier models cost more per token but raise the accuracy ceiling.	Run claude-haiku / gpt-4o-mini by default. Reserve claude-opus / gpt-4o for adversarial or low-confidence cases.
Fallback + escalation	FLYDOCS_FALLBACK_MODEL covers transient failures; FLYDOCS_ESCALATION_MODEL covers judge-driven retries.	Set fallback to a cheaper model on a different provider to avoid double-paying for failed calls; use escalation for accuracy-when-needed without paying for it on every request.

Per-request profile

A representative production request — a Spanish notarial deed, 21 pages, with splitter + classifier + extractor + judge + visual authenticity + rule engine enabled — fires around 27 LLM calls and lands at roughly \$3 of estimated LLM cost on Anthropic Claude Opus 4-7. The same request on Sonnet 4-6 lands at roughly a third of that. On Haiku 4-5 with escalation reserved for judge failures, the average drops by another factor of three.

Cost forensics

Every response carries an aggregated usage block with total tokens, total estimated cost, breakdown by agent (extractor, classifier, splitter, judge, visual, content, rule-engine), and breakdown by model. The same per-call data is emitted on the outbound_call log lines, so spend forensics — "which agent costs us the most? which model fires for which tenant?" — is a single grep, with no need to parse the response payload.

```
outbound_call target=anthropic op=extract status=ok
latency_ms=21352 model=anthropic:claude-opus-4-7
correlation_id=3d530f07-...
in_tokens=12500 out_tokens=850 total_tokens=13350
cost_usd=0.251250
```

Operational profile

Dimension	Default	Override
Sync request budget	60 s (FLYDOCS_SYNC_TIMEOUT_S).	Bump for slower providers; or move long-running flows to the async API.
Async request budget	1800 s (FLYDOCS_ASYNC_TIMEOUT_S).	Must exceed the sum of all enabled inline stages.
Job max attempts	3 (FLYDOCS_JOB_MAX_ATTEMPTS).	Tune against the operator's tolerance for retried LLM spend on transient provider errors.
Webhook max attempts	5 (FLYDOCS_WEBHOOK_MAX_ATTEMPTS).	Receiver SLOs drive this. The publisher retries on 5xx and 429 only; 4xx is permanent.
Document size cap	32 MiB (FLYDOCS_MAX_BYTES).	Raise for trusted ingest paths; lower for public-facing endpoints.
Sync page cap	10 pages (FLYDOCS_MAX_SYNC_PAGES).	Larger documents route to the async API automatically.

CHAPTER 19

Conclusion

flydocs is the bet that intelligent document processing has moved past the OCR-and-templates era — and that the right shape for the next generation is not a managed cloud service owned by one provider but a self-contained, deployable, audit-friendly service that treats the multimodal LLM as a primitive.

The hardest engineering inside flydocs is not the LLM call. It is the surrounding disciplines — the geometric bbox validator that catches the LLM's lazy boxes; the value matcher that grounds coordinates against the real text layer; the deterministic checkers that turn an extracted string into a verified IBAN; the judge that re-grades every field against the source; the rule engine that turns extracted data into auditable decisions; the durable EDA backbone that survives worker restarts; the cost telemetry that lets operators understand their spend per agent and per model; the prompt-caching middleware that knows the difference between Anthropic and OpenAI and Google. Each discipline pays for itself the first time it catches something a naïve extractor would have missed.

What flydocs ships is the integration layer between modern multimodal AI and the operational disciplines that production back offices already trust. The service is pure-multimodal at the model layer, deterministic at the validation layer, audited at the judge layer, decisive at the rule layer, and operable at the framework layer. The result: an HTTP service that turns documents into decisions, ready to be consumed by any back-office platform that wants structured, verifiable data — and ready to extend, swap, and upgrade as the multimodal frontier moves.

About flydocs

flydocs is the document-intelligence module of **Firefly OperationOS** — the open back-office plane. As a stand-alone service it drops into any platform that needs verifiable, decisioned data from unstructured documents. Source: github.com/firefly-operationOS/flydocs.

About Firefly OperationOS

Firefly OperationOS is the open back-office plane — a set of self-contained services and a shared framework that turn every common operations task (document intelligence, messaging, scheduling, observability, identity) into callable, audit-friendly building blocks. The platform is agnostic to vertical (banking, insurance, government, logistics) and to deployment topology (single-tenant SaaS, on-prem, regulated cloud). flydocs is one of the first services in the suite; new services follow the same Firefly Framework conventions described in this paper.

Copyright © 2026 Firefly Software Solutions Inc.

Apache-2.0 licensed. Plugin extensions available under separate commercial terms.

Documentation: docs.flydocs.dev — Repository: github.com/firefly-operationOS/flydocs